BEACONHOUSE NATIONAL UNIVERSITY

FINAL EXAM REPORT

Name: Saad Mughal

BNU ID: F2023-009

Total Marks: /40
Due Date: 18 Jun, 2025

Lab Manual

For

AI-LAB(SEC_B)

Course Instructor	Ms. Noreen Khalid
Lab Instructor	Sir Osama Tariq
Semester	Spring 2025

Final Report

Cell 1: Library Imports and Environment Setup

Description: This segment imports all essential libraries required for the deep learning pipeline including TensorFlow, Keras, scikit-learn, and visualization tools. It also verifies GPU availability to ensure optimal training performance on the T4 runtime.

Key Components: TensorFlow/Keras for model building, sklearn for evaluation metrics, matplotlib/seaborn for visualization Screenshot Focus:

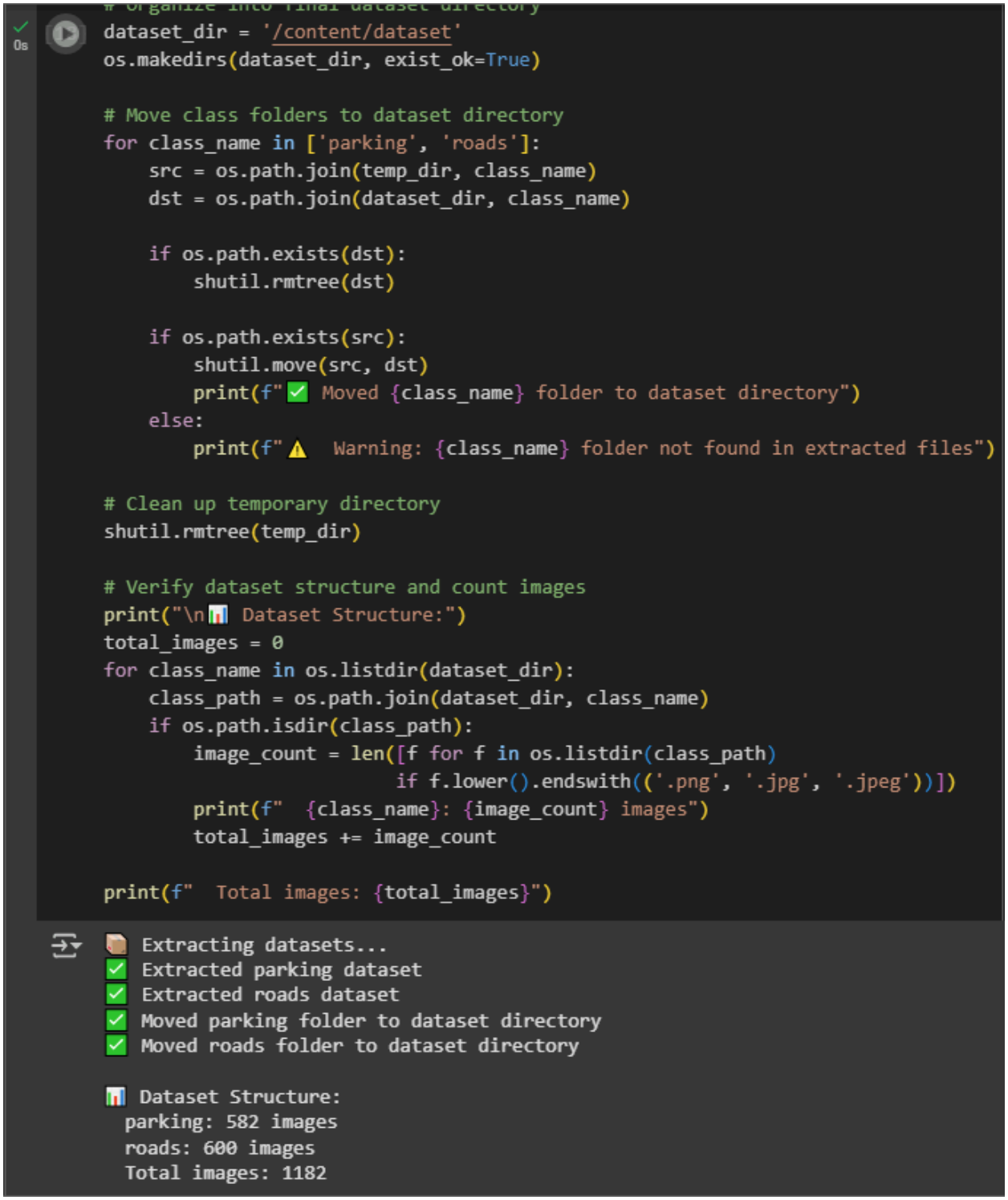
```
[14] # ================================================================================
     # DEEP LEARNING FINAL EXAM - BINARY IMAGE CLASSIFICATION
     # Parts A, B, C: Model Architecture, Regularization, Fine-Tuning
     # ================================================================================

     # Cell 1: Import Required Libraries
     # ================================================================================
     import zipfile
     import os
     import shutil
     import numpy as np
     import matplotlib.pyplot as plt
     import tensorflow as tf
     from tensorflow.keras.preprocessing import image_dataset_from_directory
     from tensorflow.keras import layers, models, regularizers
     from tensorflow.keras.applications import ResNet50
     from tensorflow.keras.callbacks import EarlyStopping
     from sklearn.metrics import classification_report, confusion_matrix
     import seaborn as sns

[15] print("TensorFlow version:", tf.__version__)
     print("GPU Available:", tf.config.list_physical_devices('GPU'))

     TensorFlow version: 2.18.0
     GPU Available: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

Library import statements and GPU detection output

Cell 2: Dataset Extraction and Organization

Description: Extracts the parking and roads image datasets from ZIP files and organizes them into a proper directory structure suitable for TensorFlow's image_dataset_from_directory function. The code creates a systematic folder hierarchy and provides dataset statistics including image counts per class.

Key Components: ZIP extraction, directory organization, dataset structure verification
Screenshot Focus:



```
# Organize into final dataset directory
dataset_dir = '/content/dataset'
os.makedirs(dataset_dir, exist_ok=True)

# Move class folders to dataset directory
for class_name in ['parking', 'roads']:
    src = os.path.join(temp_dir, class_name)
    dst = os.path.join(dataset_dir, class_name)

    if os.path.exists(dst):
        shutil.rmtree(dst)

    if os.path.exists(src):
        shutil.move(src, dst)
        print(f"✅ Moved {class_name} folder to dataset directory")
    else:
        print(f"⚠️  Warning: {class_name} folder not found in extracted files")

# Clean up temporary directory
shutil.rmtree(temp_dir)

# Verify dataset structure and count images
print("\n📊 Dataset Structure:")
total_images = 0
for class_name in os.listdir(dataset_dir):
    class_path = os.path.join(dataset_dir, class_name)
    if os.path.isdir(class_path):
        image_count = len([f for f in os.listdir(class_path)
                           if f.lower().endswith(('.png', '.jpg', '.jpeg'))])
        print(f"  {class_name}: {image_count} images")
        total_images += image_count

print(f"  Total images: {total_images}")
```

```
📦 Extracting datasets...
✅ Extracted parking dataset
✅ Extracted roads dataset
✅ Moved parking folder to dataset directory
✅ Moved roads folder to dataset directory

📊 Dataset Structure:
  parking: 582 images
  roads: 600 images
  Total images: 1182
```

Dataset extraction process and final folder structure with image counts

---

Cell 3: Data Loading and Preprocessing

Description: Creates training, validation, and test datasets using TensorFlow's image_dataset_from_directory with an 80-20 split. Configures optimal parameters including image size (224×224 for ResNet50), batch size (32 for T4 GPU), and implements dataset caching and prefetching for improved performance.

Key Components: Dataset splitting, image resizing, batch configuration, performance optimization Screenshot Focus:

```
[31] 0s
    label_mode='binary'  # Binary classification (0: parking, 1: roads)
)

# ✅ ADD THIS LINE TO SAVE CLASS NAMES
class_names = train_ds.class_names

# Create validation dataset (20% of data, split further for test set)
val_ds = image_dataset_from_directory(
    dataset_dir,
    validation_split=0.2,
    subset="validation",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode='binary'
)

# Create test set from validation data (take 20% of validation for testing)
test_ds = val_ds.take(len(val_ds) // 5)
val_ds = val_ds.skip(len(val_ds) // 5)

# Display class information
print(f"✅ Class names: {train_ds.class_names}")
print(f"✅ Training batches: {len(train_ds)}")
print(f"✅ Validation batches: {len(val_ds)}")
print(f"✅ Test batches: {len(test_ds)}")

# Optimize dataset performance
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.cache().prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)
test_ds = test_ds.cache().prefetch(buffer_size=AUTOTUNE)
```

```
Creating datasets...
Found 1182 files belonging to 2 classes.
Using 946 files for training.
Found 1182 files belonging to 2 classes.
Using 236 files for validation.
✅ Class names: ['parking', 'roads']
✅ Training batches: 30
✅ Validation batches: 7
✅ Test batches: 1
```

Dataset creation output showing batch counts and class names

Cell 4: Data Augmentation Implementation

Description: Implements the first regularization technique by creating a data augmentation pipeline. This includes horizontal flips, random rotations (±10%), and random zoom (±10%) to artificially expand the dataset and improve model generalization by exposing it to varied image transformations.

Key Components: RandomFlip, RandomRotation, RandomZoom layers, augmentation visualization Screenshot Focus:

```
[32] # Cell 4: Data Augmentation (Regularization Technique #1)
     # ================================================================================
     """
     PART B - REGULARIZATION TECHNIQUE 1: DATA AUGMENTATION
     - Horizontal flips to increase dataset diversity
     - Random rotations to improve model robustness
     - Random zoom to handle scale variations
     - Helps prevent overfitting by artificially expanding the dataset
     """

     print("🎨 Setting up data augmentation...")

     data_augmentation = tf.keras.Sequential([
         layers.RandomFlip("horizontal"),           # Mirror images horizontally
         layers.RandomRotation(0.1),               # Rotate images by ±10%
         layers.RandomZoom(0.1),                   # Zoom in/out by ±10%
     ], name="data_augmentation")

     print("✅ Data augmentation pipeline created")

     # Visualize augmentation effects (optional)
     def visualize_augmentation():
         plt.figure(figsize=(12, 8))
         for images, labels in train_ds.take(1):
             for i in range(6):
                 ax = plt.subplot(2, 3, i + 1)
                 augmented_image = data_augmentation(tf.expand_dims(images[0], 0))
                 plt.imshow(augmented_image[0].numpy().astype("uint8"))
                 plt.title(f"Augmented Image {i+1}")
                 plt.axis("off")
         plt.suptitle("Data Augmentation Examples")
         plt.tight_layout()
         plt.show()

     # Uncomment to see augmentation examples
     # visualize_augmentation()

     🎨 Setting up data augmentation...
     ✅ Data augmentation pipeline created
```

Data augmentation pipeline creation and sample augmented images

Cell 5: Model Architecture Construction (Part A)

Description: Builds the complete model architecture as required in Part A. Loads pre-trained ResNet50 with frozen layers, adds a custom classification head featuring GlobalAveragePooling2D, BatchNormalization, Dropout layers (0.3 and 0.2), Dense layer with L2 regularization, and binary output layer with sigmoid activation.

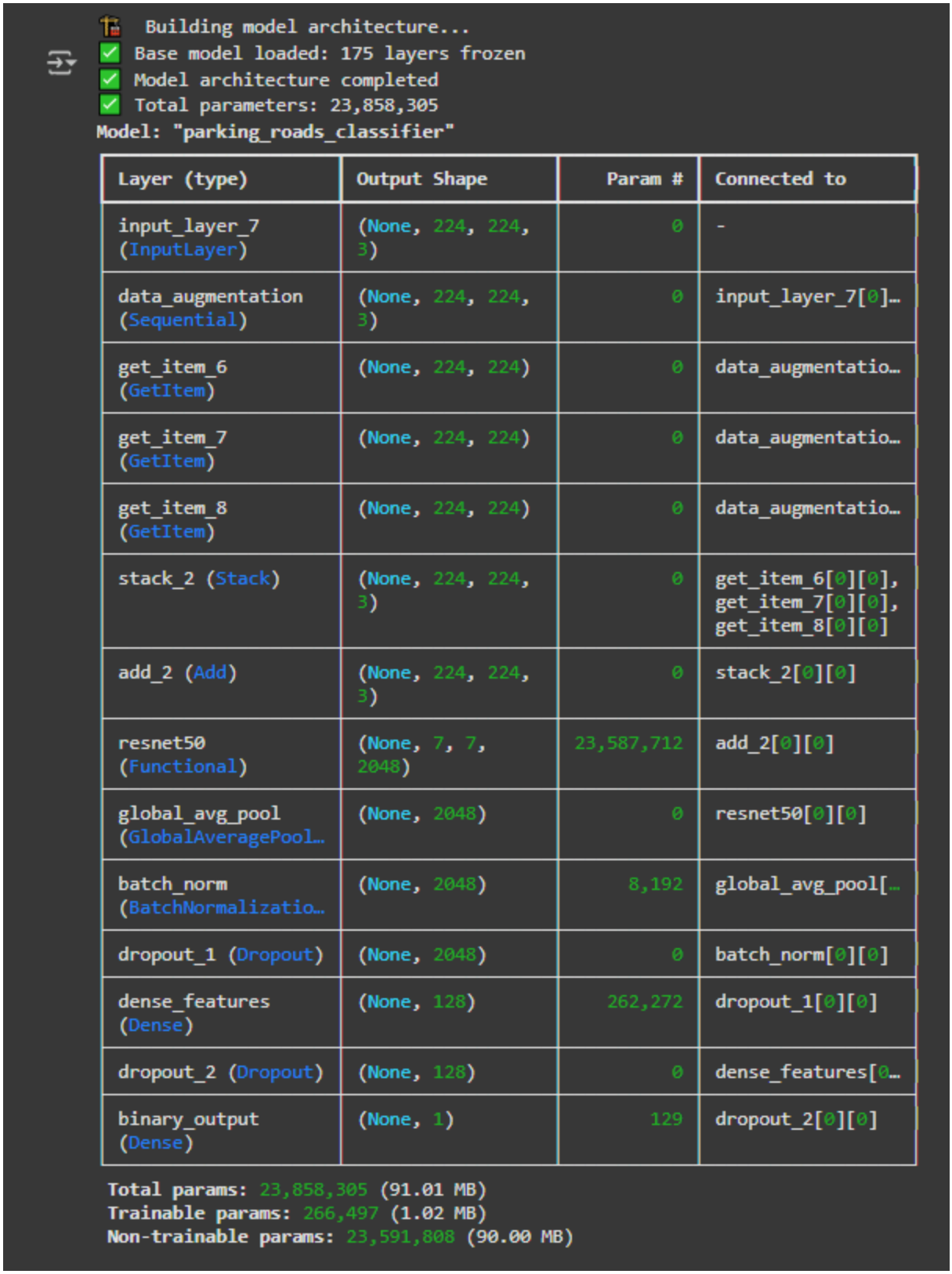
Key Components: ResNet50 base model, custom classification head, multiple regularization layers Screenshot Focus:

```
[33] # Cell 5: Model Architecture Setup (Part A)
     # ================================================================================
     """
     PART A - MODEL ARCHITECTURE:
     1. Load pre-trained ResNet50 (ImageNet weights)
     2. Freeze base model layers to retain learned features
     3. Add custom classification head with:
        - GlobalAveragePooling2D
        - BatchNormalization (Regularization #2)
        - Dropout layers (Regularization #3)
        - Dense layers with L2 regularization (Regularization #4)
        - Binary output layer with sigmoid activation
     """

     print("🏗️  Building model architecture...")

     # 1. Load pre-trained ResNet50 base model
     base_model = ResNet50(
         weights='imagenet',        # Use ImageNet pre-trained weights
         include_top=False,         # Exclude final classification layer
         input_shape=IMG_SIZE + (3,)  # RGB images of size 224x224
     )

     # 2. Freeze base model layers (Part A requirement)
     base_model.trainable = False
     print(f"✅ Base model loaded: {len(base_model.layers)} layers frozen")

     # 3. Build custom classification head
     inputs = tf.keras.Input(shape=IMG_SIZE + (3,))

     # Apply data augmentation
     x = data_augmentation(inputs)

     # Preprocess for ResNet50
     x = tf.keras.applications.resnet50.preprocess_input(x)

     # Base model feature extraction
     x = base_model(x, training=False)
```

```python
# Custom classification head
x = layers.GlobalAveragePooling2D(name="global_avg_pool")(x)  # Part A requirement

# REGULARIZATION TECHNIQUE #2: Batch Normalization
x = layers.BatchNormalization(name="batch_norm")(x)

# REGULARIZATION TECHNIQUE #3: Dropout (≤ 0.4 as required)
x = layers.Dropout(0.3, name="dropout_1")(x)

# Dense layer with L2 regularization (REGULARIZATION TECHNIQUE #4)
x = layers.Dense(
    128,
    activation='relu',
    kernel_regularizer=regularizers.l2(0.01),  # L2 weight decay
    name="dense_features"
)(x)

# Additional dropout for extra regularization
x = layers.Dropout(0.2, name="dropout_2")(x)

# Binary output layer (Part A requirement)
outputs = layers.Dense(1, activation='sigmoid', name="binary_output")(x)

# Create final model
model = models.Model(inputs, outputs, name="parking_roads_classifier")

print("✅ Model architecture completed")
print(f"✅ Total parameters: {model.count_params():,}")

# Display model summary
model.summary()
```

```
🏗️ Building model architecture...
✅ Base model loaded: 175 layers frozen
✅ Model architecture completed
✅ Total parameters: 23,858,305
Model: "parking_roads_classifier"
```

Layer (type)	Output Shape	Param #	Connected to
input_layer_7 (InputLayer)	(None, 224, 224, 3)	0	-
data_augmentation	(None, 224, 224,	0	input_layer_7[0]…

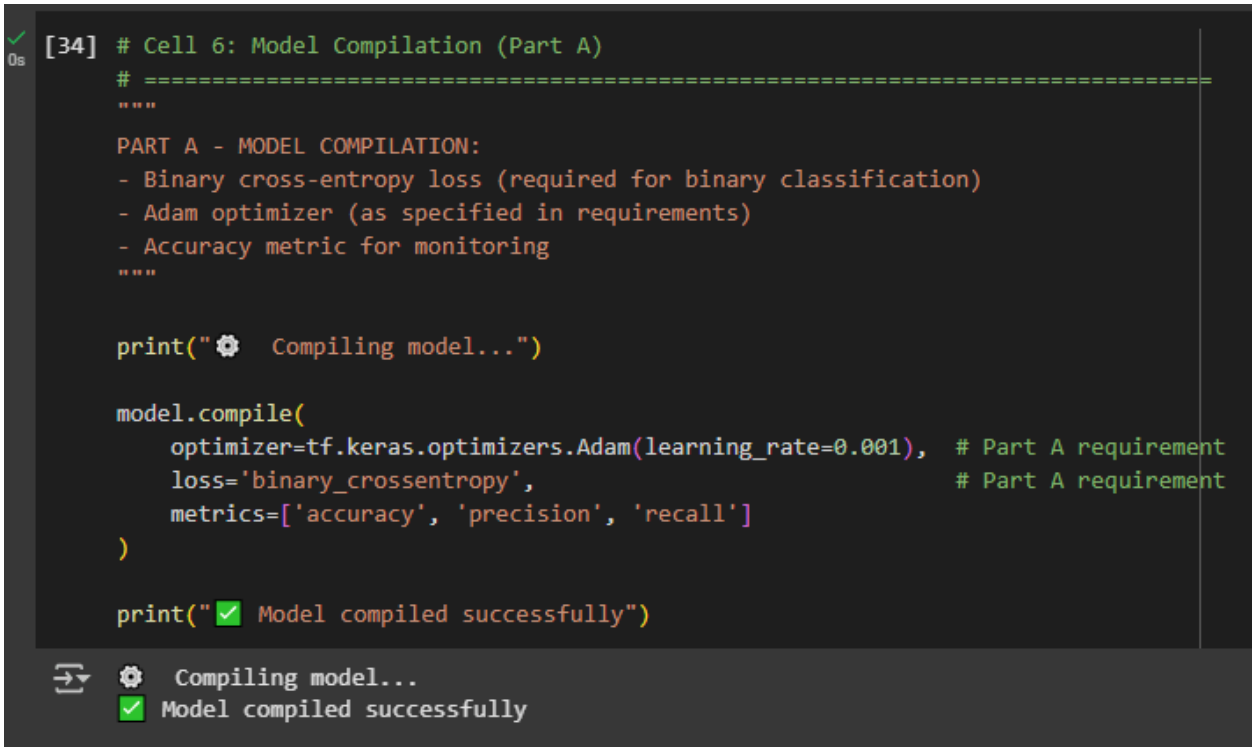


Model architecture code and model.summary() output showing layer structure

Cell 6: Model Compilation

Description: Compiles the model according to Part A requirements using binary cross-entropy loss function (appropriate for binary classification) and Adam optimizer. Additional metrics including precision and recall are included for comprehensive performance monitoring.

Key Components: Binary cross-entropy loss, Adam optimizer, evaluation metrics Screenshot Focus:



```
[34] # Cell 6: Model Compilation (Part A)
     # ================================================================================
     """
     PART A - MODEL COMPILATION:
     - Binary cross-entropy loss (required for binary classification)
     - Adam optimizer (as specified in requirements)
     - Accuracy metric for monitoring
     """

     print("⚙️  Compiling model...")

     model.compile(
         optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),  # Part A requirement
         loss='binary_crossentropy',                               # Part A requirement
         metrics=['accuracy', 'precision', 'recall']
     )

     print("✅ Model compiled successfully")

     ⚙️  Compiling model...
     ✅ Model compiled successfully
```

Model compilation code and confirmation message

Cell 7: Initial Training with Early Stopping

Description: Executes the initial training phase for 5 epochs with frozen base layers as specified in the exam requirements. Implements Early Stopping regularization technique that monitors validation loss with patience=3 to prevent overfitting and restore best weights automatically.

Key Components: 5-epoch training, Early Stopping callback, frozen layer training Screenshot Focus:

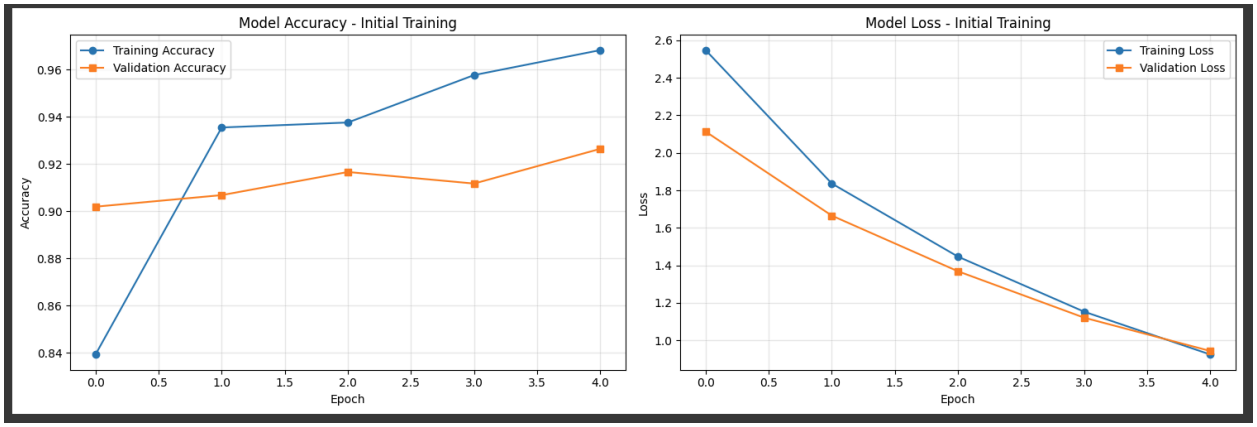
```
[35] # Cell 7: Initial Training with Early Stopping (Part A & B)
     # ============================================================================
     """
     PART A & B - INITIAL TRAINING:
     - Train for 5 epochs with frozen base layers
     - REGULARIZATION TECHNIQUE #5: Early Stopping to prevent overfitting
     - Monitor validation loss with patience=3
     - Restore best weights when stopped early
     """

     print("🚀 Starting initial training (5 epochs with frozen layers)...")

     # REGULARIZATION TECHNIQUE #5: Early Stopping
     early_stopping = EarlyStopping(
         monitor='val_loss',           # Monitor validation loss
         patience=3,                   # Wait 3 epochs before stopping
         restore_best_weights=True,    # Restore best model weights
         verbose=1,
         mode='min'
     )

     # Train the model
     history = model.fit(
         train_ds,
         validation_data=val_ds,
         epochs=5,                     # Part A requirement: 5 epochs
         callbacks=[early_stopping],
         verbose=1
     )

     print("✅ Initial training completed")

🚀 Starting initial training (5 epochs with frozen layers)...
Epoch 1/5
30/30 ━━━━━━━━━━━━━━━━━━━━ 16s 279ms/step - accuracy: 0.7567 - loss: 2.8151 - precision: 0.7392 - recall: 0.7977 - val_accuracy: 0.9020 - val_loss: 2.1133 - val_precision: 0.8571 - val_recall: 0.9913
Epoch 2/5
30/30 ━━━━━━━━━━━━━━━━━━━━ 5s 161ms/step - accuracy: 0.9357 - loss: 1.9379 - precision: 0.9206 - recall: 0.9510 - val_accuracy: 0.9069 - val_loss: 1.6652 - val_precision: 0.8692 - val_recall: 0.9826
Epoch 3/5
30/30 ━━━━━━━━━━━━━━━━━━━━ 5s 162ms/step - accuracy: 0.9288 - loss: 1.5241 - precision: 0.9285 - recall: 0.9255 - val_accuracy: 0.9167 - val_loss: 1.3683 - val_precision: 0.8828 - val_recall: 0.9826
Epoch 4/5
30/30 ━━━━━━━━━━━━━━━━━━━━ 5s 162ms/step - accuracy: 0.9549 - loss: 1.2245 - precision: 0.9433 - recall: 0.9660 - val_accuracy: 0.9118 - val_loss: 1.1208 - val_precision: 0.8760 - val_recall: 0.9826
Epoch 5/5
30/30 ━━━━━━━━━━━━━━━━━━━━ 5s 165ms/step - accuracy: 0.9608 - loss: 0.9818 - precision: 0.9624 - recall: 0.9578 - val_accuracy: 0.9265 - val_loss: 0.9431 - val_precision: 0.8906 - val_recall: 0.9913
Restoring model weights from the end of the best epoch: 5.
✅ Initial training completed
```

Training execution and epoch-by-epoch progress output

Cell 8: Training Visualization

Description: Creates comprehensive visualization of the training process by plotting accuracy and loss curves for both training and validation sets. This analysis helps identify potential overfitting and validates the effectiveness of regularization techniques implemented.

Key Components: Matplotlib plotting, training history analysis, performance visualization
Screenshot Focus:

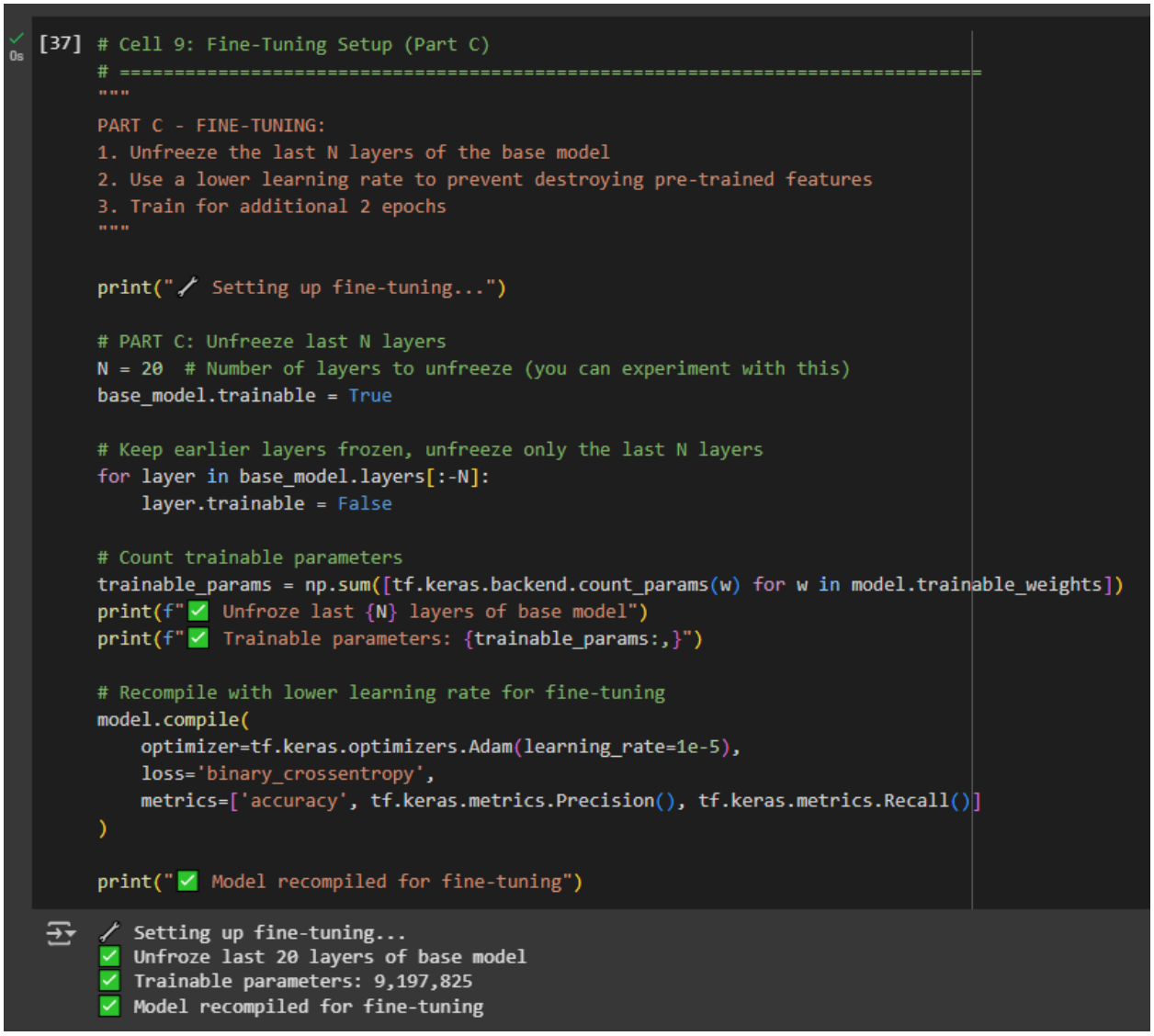


Generated accuracy and loss curve plots

Cell 9: Fine-Tuning Setup (Part C)

Description: Prepares the model for fine-tuning by unfreezing the last 20 layers of the ResNet50 base model while keeping earlier layers frozen. Recompiles the model with a significantly lower learning rate (1e-5) to prevent destroying pre-trained features during fine-tuning.

Key Components: Layer unfreezing strategy, learning rate adjustment, parameter counting
Screenshot Focus:



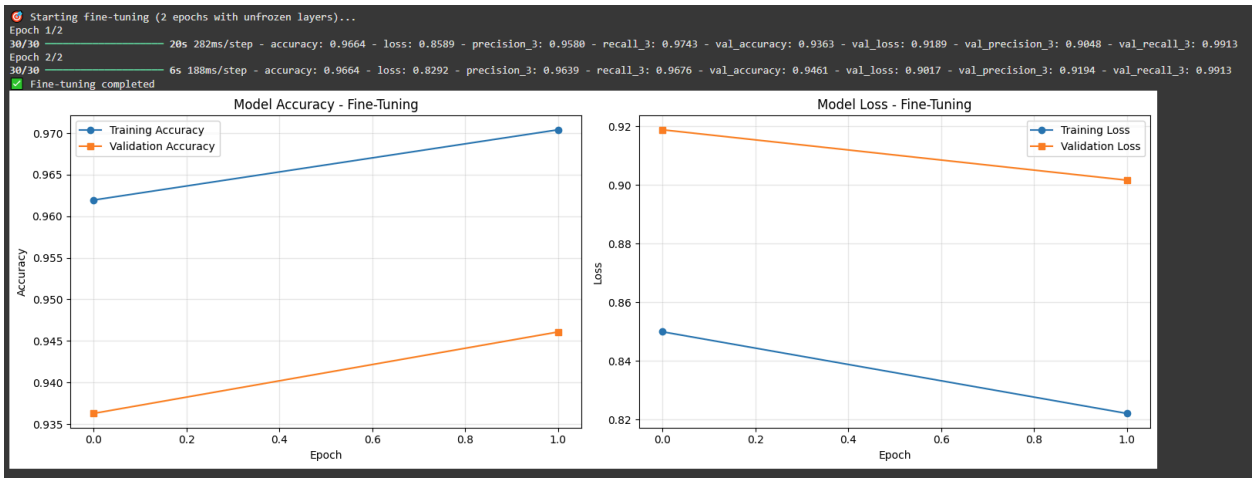
```
[37] # Cell 9: Fine-Tuning Setup (Part C)
     # ================================================================================
     """
     PART C - FINE-TUNING:
     1. Unfreeze the last N layers of the base model
     2. Use a lower learning rate to prevent destroying pre-trained features
     3. Train for additional 2 epochs
     """

     print("🔧 Setting up fine-tuning...")

     # PART C: Unfreeze last N layers
     N = 20  # Number of layers to unfreeze (you can experiment with this)
     base_model.trainable = True

     # Keep earlier layers frozen, unfreeze only the last N layers
     for layer in base_model.layers[:-N]:
         layer.trainable = False

     # Count trainable parameters
     trainable_params = np.sum([tf.keras.backend.count_params(w) for w in model.trainable_weights])
     print(f"✅ Unfroze last {N} layers of base model")
     print(f"✅ Trainable parameters: {trainable_params:,}")

     # Recompile with lower learning rate for fine-tuning
     model.compile(
         optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
         loss='binary_crossentropy',
         metrics=['accuracy', tf.keras.metrics.Precision(), tf.keras.metrics.Recall()]
     )

     print("✅ Model recompiled for fine-tuning")

🔧 Setting up fine-tuning...
✅ Unfroze last 20 layers of base model
✅ Trainable parameters: 9,197,825
✅ Model recompiled for fine-tuning
```

Fine-tuning setup code and trainable parameter statistics

Cell 10: Fine-Tuning Training

Description: Executes the fine-tuning phase for 2 additional epochs as required in Part C. Uses the unfrozen layers with reduced learning rate to adapt the pre-trained features to the specific parking vs roads classification task while preserving learned representations.

Key Components: 2-epoch fine-tuning, lower learning rate training, performance monitoring
Screenshot Focus:

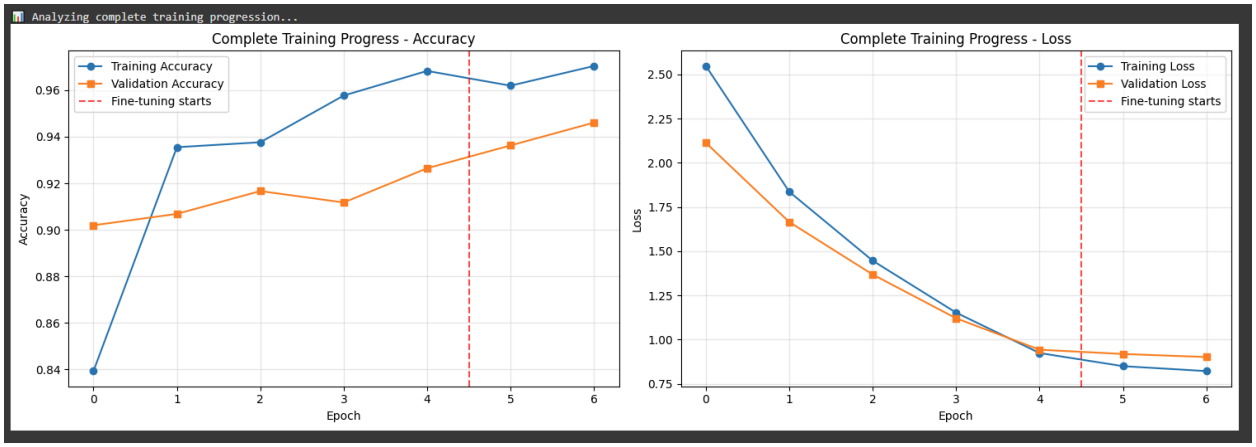


Fine-tuning training progress and epoch results

Cell 11: Combined Training Analysis

Description: Provides comprehensive analysis by combining both training phases (initial + fine-tuning) into unified visualization. Shows the complete training progression with a vertical line indicating where fine-tuning begins, demonstrating the impact of unfreezing layers.

Key Components: Combined history plotting, training phase demarcation, complete progress analysis
Screenshot Focus:

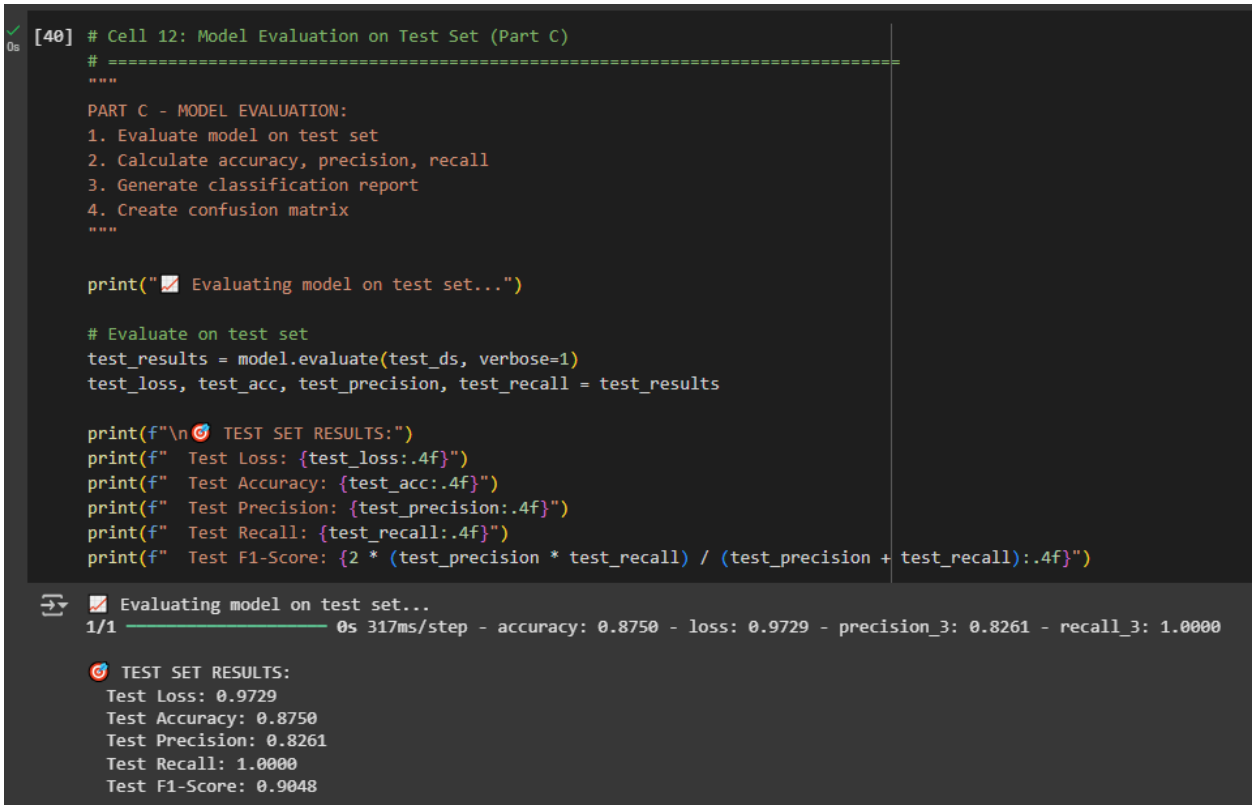


Combined training curves showing both phases with fine-tuning boundary

Cell 12: Test Set Evaluation

Description: Evaluates the final trained model on the reserved test set to provide unbiased performance metrics. Calculates key classification metrics including accuracy, precision, recall, and F1-score to comprehensively assess model performance on unseen data.

Key Components: Test set evaluation, performance metrics calculation, final results summary Screenshot Focus:



```
[40] # Cell 12: Model Evaluation on Test Set (Part C)
     # ========================================================================
     """
     PART C - MODEL EVALUATION:
     1. Evaluate model on test set
     2. Calculate accuracy, precision, recall
     3. Generate classification report
     4. Create confusion matrix
     """

     print("📈 Evaluating model on test set...")

     # Evaluate on test set
     test_results = model.evaluate(test_ds, verbose=1)
     test_loss, test_acc, test_precision, test_recall = test_results

     print(f"\n🎯 TEST SET RESULTS:")
     print(f"  Test Loss: {test_loss:.4f}")
     print(f"  Test Accuracy: {test_acc:.4f}")
     print(f"  Test Precision: {test_precision:.4f}")
     print(f"  Test Recall: {test_recall:.4f}")
     print(f"  Test F1-Score: {2 * (test_precision * test_recall) / (test_precision + test_recall):.4f}")

     📈 Evaluating model on test set...
     1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 317ms/step - accuracy: 0.8750 - loss: 0.9729 - precision_3: 0.8261 - recall_3: 1.0000

     🎯 TEST SET RESULTS:
       Test Loss: 0.9729
       Test Accuracy: 0.8750
       Test Precision: 0.8261
       Test Recall: 1.0000
       Test F1-Score: 0.9048
```

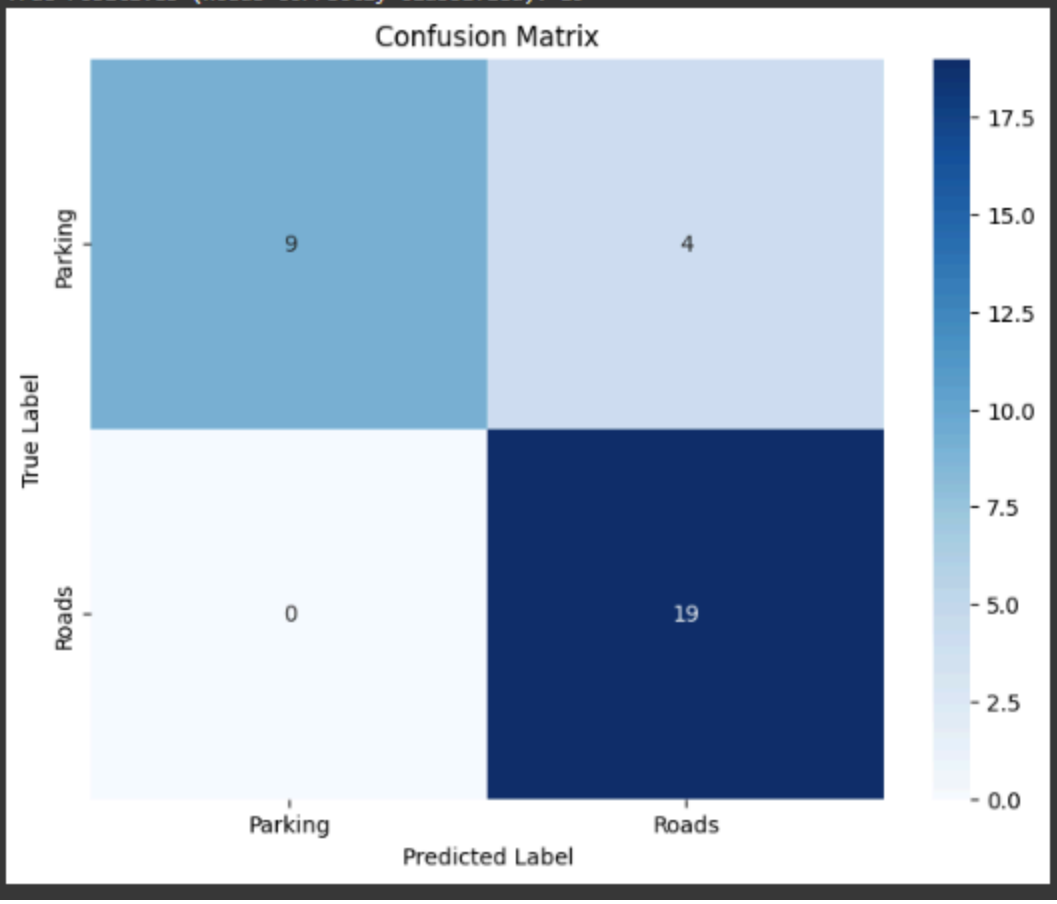
Model evaluation output with all performance metrics

Cell 13: Classification Analysis (Part C)

Description: Generates detailed classification analysis as required in Part C including a comprehensive classification report with per-class metrics and confusion matrix. Creates professional visualization of the confusion matrix using seaborn heatmap to clearly show model performance across both classes.

Key Components: Classification report, confusion matrix, performance visualization, detailed metrics Screenshot Focus:

```
🔍 Generating detailed classification analysis...
Generating predictions...

✅ Generated predictions for 32 test samples

📊 CLASSIFICATION REPORT:
==================================================
              precision    recall  f1-score   support

     Parking       1.00      0.69      0.82        13
       Roads       0.83      1.00      0.90        19

    accuracy                           0.88        32
   macro avg       0.91      0.85      0.86        32
weighted avg       0.90      0.88      0.87        32


🎯 CONFUSION MATRIX:
==============================
True Negatives (Parking correctly classified): 9
False Positives (Parking misclassified as Roads): 4
False Negatives (Roads misclassified as Parking): 0
True Positives (Roads correctly classified): 19
```

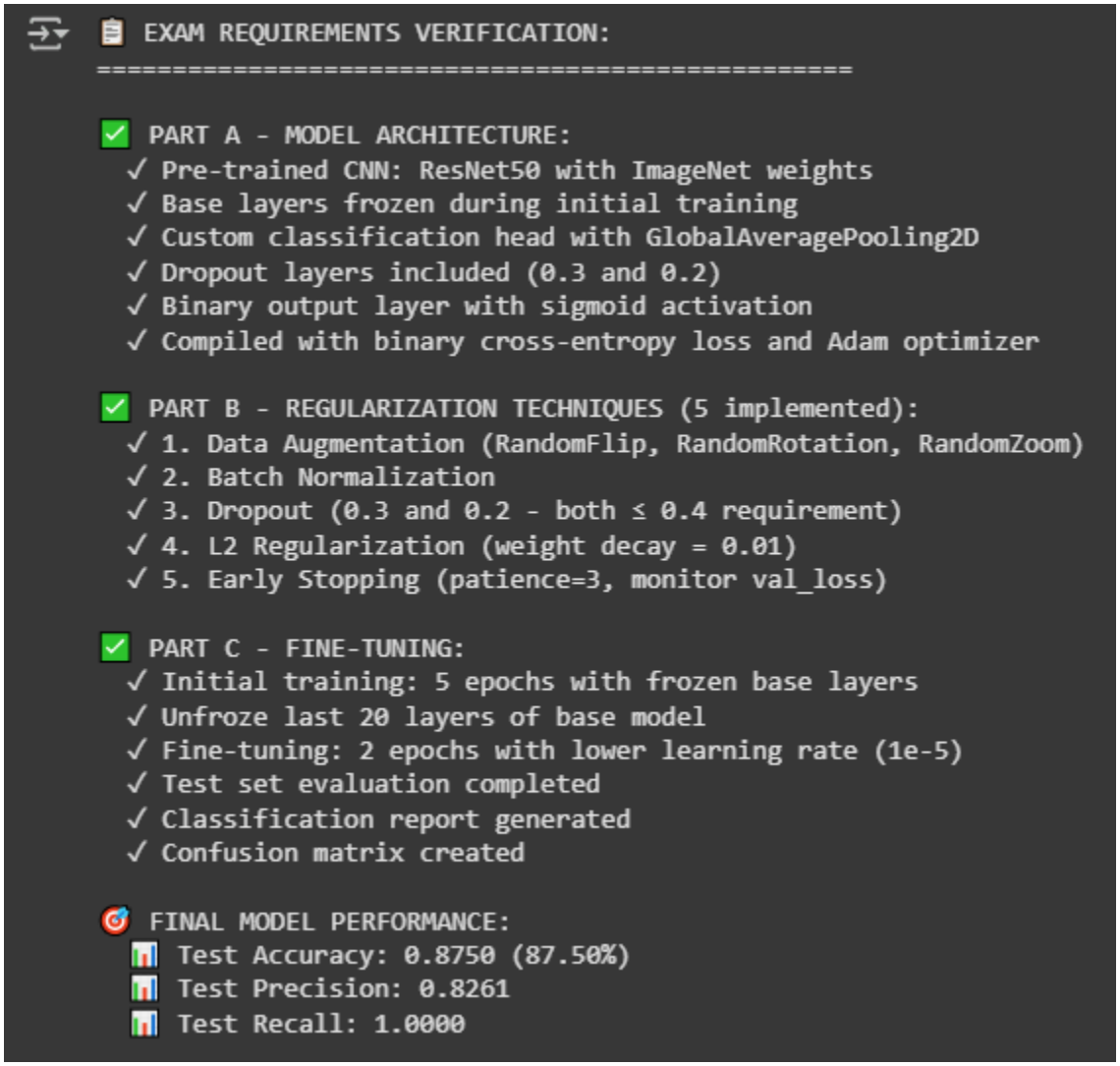


Classification report output and confusion matrix heatmap

Cell 14: Requirements Verification Summary

Description: Provides comprehensive verification that all exam requirements for Parts A, B, and C have been successfully implemented. Documents all architectural choices, regularization techniques used, and training procedures followed, serving as a complete checklist for exam compliance.

Key Components: Requirements checklist, implementation verification, final performance summary Screenshot Focus:



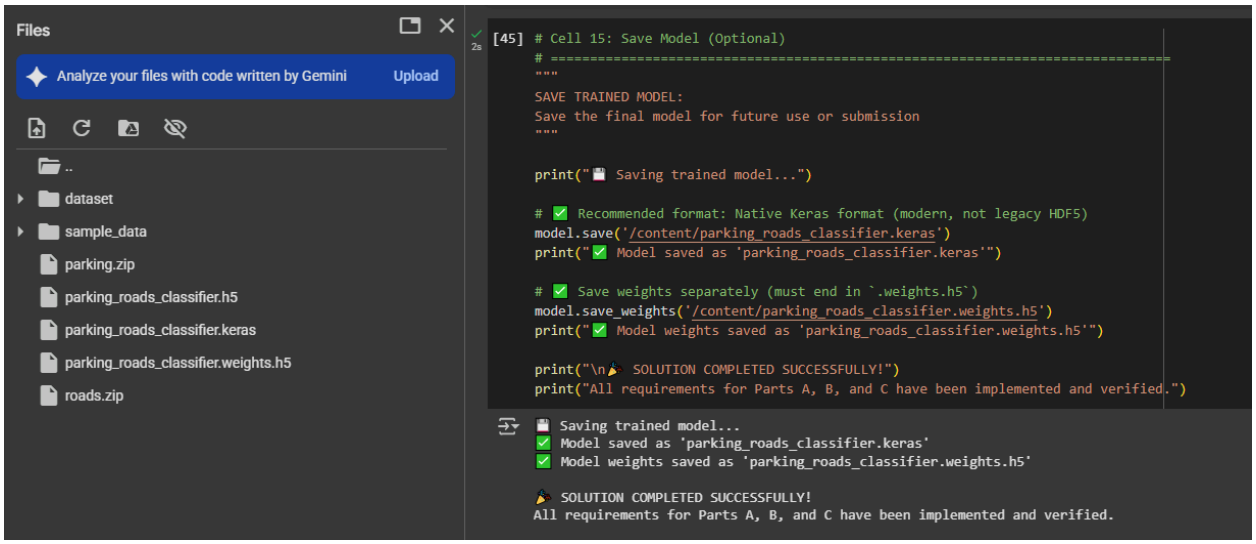
Complete requirements verification output with checkmarks and final metrics

---

Cell 15: Model Saving

Description: Saves the trained model in standard formats (.h5) for future use or submission. Provides both complete model saving and weights-only saving options, ensuring the trained model can be preserved and loaded for further analysis or deployment.

Key Components: Model serialization, file saving, preservation for future use Screenshot Focus:



Model saving confirmation messages and file locations.